The Project Blueprint: Tropical Reserve Rescue

Set the scene in a Sri Lankan eco-tourism wildlife reserve (like Sinharaja). A severe storm has caused mudslides and knocked down massive trees. Autonomous rescue drones (the AI agents) are deployed to find stranded hikers or endangered animals and guide them back to base.

The drones fly automatically using pathfinding logic, but the fallen trees block their way. The player drives a monster truck through the jungle, using the truck's momentum or a winch to physically pull the heavy logs out of the way. When a log moves, the AI realizes the path is clear and dynamically changes its route to fly through the newly opened area.

Student 1: The World & Graph Architect

Your Goal: Build the beautiful 3D jungle and turn it into a mathematical grid for the AI.

- **Graphics Duty (Level Design & Lighting):** You are the World Builder. You will use Unity's Terrain tool to sculpt the reserve. Apply dirt and grass textures, paint the trees and rocks you imported earlier, and set up a moody, stormy lighting environment (dark clouds, maybe some fog). You will also bake the initial Unity NavMesh.
- **Intelligent Systems Duty (Graph Formulation):** You must write a C# script that programmatically extracts the 3D world data into a mathematical graph (nodes and adjacency lists).
- **Your Action Guide:**
    1. Build the visual terrain first so you have something to work with.
    2. Instead of letting the AI just use Unity's built-in NavMesh, write a script that casts invisible rays (Raycasts) down from the sky in a grid pattern.
    3. If the ray hits the ground, create a "Walkable Node". If it hits a tree or rock, create an "Unwalkable Node". Store these nodes in a 2D Array or Adjacency List. This is your custom graph!

Student 2: The Physics & Adaptation Engineer

Your Goal: Make the world interactive and tell the math graph when things change.

- **Graphics Duty (Player Interaction Physics):** You are the Systems Engineer. You are responsible for programming the physics and scripts that allow the human player to move barricades. This means coding the monster truck to drive smoothly and setting up heavy Rigidbody physics on the fallen logs so the truck can ram them or winch them out of the way.
- **Intelligent Systems Duty (Dynamic Adaptation):** You handle the logic that detects when the player alters the environment and severs the mathematical edges in the graph.
- **Your Action Guide:**

1. Get the truck driving controls feeling heavy and powerful. Add Box Colliders and heavy mass to the fallen logs.
2. Write an event trigger. When the player pushes a log out of a grid space, your code must tell Student 1's graph: *"Update Node [X, Y] from Unwalkable to Walkable"* and then trigger the AI to recalculate.

Student 3: The Modeler & A^* Specialist

Your Goal: Build custom 3D models from scratch and write the primary brain of the AI.

- **Graphics Duty (Custom 3D Modeling):** You are the Core Developer. You cannot use free asset store models for your specific task. You must use Blender or Maya to design, model, and import at least two custom 3D models from scratch. Model a high-tech Rescue Drone and a flashing Rescue Beacon.
- **Intelligent Systems Duty (A^* Search):** You are responsible for programming the primary A^* navigation algorithm, the priority queue, and mathematically justifying the heuristic function.
- **Your Action Guide:**
    1. Start your Blender models immediately, ensuring they have good topology and colors (UV mapping), then import them into Unity.
    2. In C#, build the A^* algorithm using the formula $f(n) = g(n) + h(n)$.
    3. Since your drone can fly over small bumps but has to avoid giant trees, use the Euclidean distance formula for your heuristic (measuring the straight line to the beacon). You will need to explain this choice during your Viva.

Student 4: The Animator & Debug Tech

Your Goal: Make the math visible, both through smooth drone animations and neon debug lines.

- **Graphics Duty (Agent Animation & Movement):** You are the Agent Controller. You take the mathematical path arrays generated by Student 3's algorithm and translate them into smooth 3D character movement and rotations in the engine.
- **Intelligent Systems Duty (Secondary Search & Debug Visualizer):** You must implement a backup search algorithm (like BFS or UCS) and program a "Toggleable Debug Mode" that draws the search frontiers directly into the 3D engine view.
- **Your Action Guide:**
    1. When the drone moves from Node A to Node B, don't let it just snap into place. Use Vector3.Lerp and Quaternion.Slerp in Unity to make it bank, tilt, and fly smoothly like a real quadcopter.
    2. Write a Breadth-First Search (BFS) as a fallback if A^* fails.
    3. Use Unity's LineRenderer component. When you press a button (like 'Tab'), it should draw glowing red lines over the graph nodes the algorithm checked, and a glowing green line for the final path the drone takes.